

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

кафедра програмних засобів

ЗВІТ

з лабораторної роботи № 3

з дисципліни «Агентно-орієнтоване програмування» на тему:

**«РОЗРОБКА АГЕНТІВ ЗІ СКЛАДНОЮ ПОВЕДІНКОЮ В PYTHON
ЗА ДОПОМОГОЮ ФРЕЙМВОРКУ SPADE»**

Виконав:

ст. гр. КНТ-113сп

Іван ЩЕДРОВСЬКИЙ

Прийняв:

Доцент

Олена ПОДКОВАЛІХІНА

1 Мета роботи

Ознайомитися з поняттям складної поведінки агентів (Complex Behaviours) у мультиагентних системах. Навчитися використовувати різні типи поведінки в SPADE: послідовну (Sequential), циклічну (Cyclic), умовну (FSM) та таймерну (Periodic). Реалізувати агентів, які можуть адаптувати свою поведінку залежно від умов середовища

2 Завдання до лабораторної роботи

Реалізувати мультиагентну систему для симуляції черги в сервісі (наприклад, черга до каси або до сервера): агенти-клієнти стають у чергу, а агент-сервіс обслуговує їх із різною швидкістю.

3 Виконання лабораторної роботи

Для симуляції черги з агентами було обрано симуляцію черги за якоюсь інформацією. Для реалізації було створено два види агентів.

Перший це сервіс, або менеджер, який приймає запити на інформацію та, через деякий випадковий час, її повертає. Цей агент не має складної логіки та його основна задача це отримувати запит на інформацію та задовільняти, або не задовільняти його. Для більшої цікавості роботи з агентами, сервіс не буде оброблювати деякі запити.

Другий це клієнт, який буде запитувати інформацію у менеджера. Цей агент буде мати декілька станів та перемикатись між ними.

На рисунку 1 показана діаграма взаємодії та переключення між станами агентів

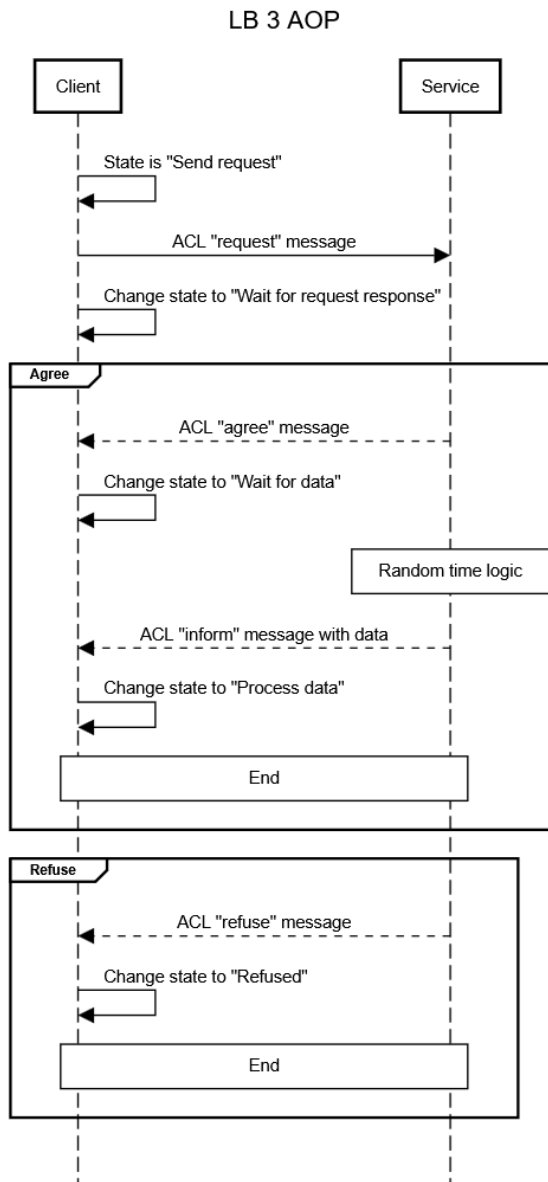


Рисунок 1 – Діаграма взаємодії агентів

На рисунках 2, 3 та 4 показано приклад запуску програми.

```
Service: Hi!

Service: Waiting for messages!
Service: Message from "ltlaitoff-2@xmpp.jp/8000445049731172983998338724" received!
Service: Send refuse message to ltlaitoff-2@xmpp.jp/8000445049731172983998338724

Service: Waiting for messages!
Service: Message from "ltlaitoff-3@xmpp.jp/12841324169065582922998339076" received!
Service: Send agree message to ltlaitoff-3@xmpp.jp/12841324169065582922998339076
[ASYNC] Service: Schedule send message to ltlaitoff-3@xmpp.jp/12841324169065582922998339076 with value 5 after 0.38 seconds

Service: Waiting for messages!
[ASYNC] Service: Send data to ltlaitoff-3@xmpp.jp/12841324169065582922998339076. Value is 5
Service: Message from "ltlaitoff-2@xmpp.jp/56004597730694679353448513059" received!
Service: Send agree message to ltlaitoff-2@xmpp.jp/56004597730694679353448513059
[ASYNC] Service: Schedule send message to ltlaitoff-2@xmpp.jp/56004597730694679353448513059 with value 7 after 1.54 seconds

Service: Waiting for messages!
Service: Message from "ltlaitoff-3@xmpp.jp/28112149971567228785954708706" received!
Service: Send agree message to ltlaitoff-3@xmpp.jp/28112149971567228785954708706
[ASYNC] Service: Schedule send message to ltlaitoff-3@xmpp.jp/28112149971567228785954708706 with value 7 after 0.86 seconds
```

Рисунок 2 – Інформація про запуск сервісу

```
Client 2.0: Hi!

FSM starting at initial state SEND_REQUEST_STATE
Client 2.0: Send request message to ltlaitoff@xmpp.jp
Client 2.0: Waiting for request response!
Client 2.0: Request response message from "ltlaitoff@xmpp.jp/136834852400599928843448388003" received!

Client 2.0: Request was refused from service!
Client 2.0: Bye!
FSM finished at state REFUSED_STATE
Client 2.1: Hi!

FSM starting at initial state SEND_REQUEST_STATE
Client 2.1: Send request message to ltlaitoff@xmpp.jp
Client 2.1: Waiting for request response!
Client 2.1: Request response message from "ltlaitoff@xmpp.jp/7589357130343396675954702626" received!

Client 2.1: Request was refused from service!
Client 2.1: Bye!
FSM finished at state REFUSED_STATE
Client 2.2: Hi!
```

Рисунок 3 – Інформація про запуск перших агентів

```

FSM starting at initial state SEND_REQUEST_STATE
Client 2.0: Send request message to ltlaitoff@xmpp.jp
Client 2.0: Waiting for request response!
Client 2.0: Request response message from "ltlaitoff@xmpp.jp/136834852400599928843448388003" received!
Client 2.0: Waiting for data!
Client 2.0: Data message from "ltlaitoff@xmpp.jp/136834852400599928843448388003" received! Data = 3
Client 2.0: Final value is 3!
Client 2.0: Bye!
FSM finished at state PROCESS_DATA_STATE
Client 2.1: Hi!

FSM starting at initial state SEND_REQUEST_STATE
Client 2.1: Send request message to ltlaitoff@xmpp.jp
Client 2.1: Waiting for request response!
Client 2.1: Request response message from "ltlaitoff@xmpp.jp/7589357130343396675954702626" received!
Client 2.1: Waiting for data!
Client 2.1: Data message from "ltlaitoff@xmpp.jp/7589357130343396675954702626" received! Data = 5
Client 2.1: Final value is 5!
Client 2.1: Bye!
FSM finished at state PROCESS_DATA_STATE
Client 2.2: Hi!

```

Рисунок 4 – Інформація про запуск других агентів

4 Тест розробленої програми

```

##client_agent.py
import random
import asyncio

from spade.template import Template
from spade.agent import Agent
from spade.behaviour import CyclicBehaviour, FSMBehaviour, State
from spade.message import Message

request_data_template = Template()
request_data_template.set_metadata("performative", "request")
request_data_template.set_metadata("language", "FIFA-SL")
request_data_template.set_metadata("ontology", "REQUEST-DATA")

agree_response_template = Template()
agree_response_template.set_metadata("performative", "agree")
agree_response_template.set_metadata("language", "FIFA-SL")
agree_response_template.set_metadata("ontology", "AGREE")

refuse_response_template = Template()
refuse_response_template.set_metadata("performative", "refuse")
refuse_response_template.set_metadata("language", "FIFA-SL")
refuse_response_template.set_metadata("ontology", "REFUSE")

data_response_template = Template()
data_response_template.set_metadata("performative", "inform")
data_response_template.set_metadata("language", "FIFA-SL")

```

```

data_response_template.set_metadata("ontology", "DATA")

class ClientAgent(Agent):
    # Visual(in console) agent name. Different from jid
    name = "Client X"

    # How long we wait for message? In seconds
    message_timeout = 10

    # With what service agent work with? JID
    service_id = None

    # Final value from sender. Property created for transfer value from one state to other
    received_value = None

    SEND_REQUEST_STATE = "SEND_REQUEST_STATE"
    WAIT_FOR_REQUEST_RESPONSE = "WAIT_FOR_REQUEST_RESPONSE"
    REFUSED_STATE = "REFUSED_STATE"
    WAIT_FOR_DATA_STATE = "WAIT_FOR_DATA_STATE"
    PROCESS_DATA_STATE = "PROCESS_DATA_STATE"

    def __init__(self, jid: str, password: str, name: str, service_id):
        super().__init__(jid, password)
        self.name = name
        self.service_id = service_id

class ExampleFSMBehaviour(FSMBehaviour):
    async def on_start(self):
        print(f'FSM starting at initial state {self.current_state}')

    async def on_end(self):
        print(f'FSM finished at state {self.current_state}')
        await self.agent.stop()

class SendRequestState(State):
    async def run(self):
        message = Message(to=self.agent.service_id)
        message.body = "Request data"
        message.set_metadata("language", "FIFA-SL")
        message.set_metadata("performative", "request")
        message.set_metadata("ontology", "REQUEST-DATA")

        assert request_data_template.match(message), "Request message should match request template"
        print(f'{self.agent.name}: Send request message to {message.to}')
        await self.send(message)

        self.set_next_state(self.agent.WAIT_FOR_REQUEST_RESPONSE)

class WaitForRequestResponseState(State):
    async def run(self):
        print(f'{self.agent.name}: Waiting for request response!')
        message = await self.receive(timeout=self.agent.message_timeout)
        if not message:
            await self.agent.bye()
            return

        print(f'{self.agent.name}: Request response message from \"{message.sender}\" received!', message)
        assert agree_response_template.match(message) or refuse_response_template.match(message), "Data
request message should match or agree, of refuse templates"

```

```

        if message.metadata["performative"] == "refuse":
            self.set_next_state(self.agent.REFUSED_STATE)
            return

        self.set_next_state(self.agent.WAIT_FOR_DATA_STATE)

class RefusedState(State):
    async def run(self):
        print(f"\n{self.agent.name}: Request was refused from service!")
        await self.agent.bye()

class WaitForDataState(State):
    async def run(self):
        print(f"{self.agent.name}: Waiting for data!")

        message = await self.receive(timeout=self.agent.message_timeout)
        if not message:
            await self.agent.bye()
            return

        print(f"{self.agent.name}: Data message from \"{message.sender}\" received! Data = {message.body}")
        assert data_response_template.match(message), "Data message should match data response template"
        self.agent.received_value = message.body
        self.set_next_state(self.agent.PROCESS_DATA_STATE)

class ProcessDataState(State):
    async def run(self):
        print(f"{self.agent.name}: Final value is {self.agent.received_value}!")
        await self.agent.bye()

    async def bye(self):
        print(f"{self.name}: Bye!")
        await self.stop()

    async def setup(self):
        print(f"{self.name}: Hi!\n")

        fsm = self.ExampleFSMBehaviour()
        fsm.add_state(name=self.SEND_REQUEST_STATE, state=self.SendRequestState(), initial=True)
        fsm.add_state(name=self.WAIT_FOR_REQUEST_RESPONSE, state=self.WaitForRequestResponseState())
        fsm.add_state(name=self.REFUSED_STATE, state=self.RefusedState())
        fsm.add_state(name=self.WAIT_FOR_DATA_STATE, state=self.WaitForDataState())
        fsm.add_state(name=self.PROCESS_DATA_STATE, state=self.ProcessDataState())

        fsm.add_transition(source=self.SEND_REQUEST_STATE, dest=self.WAIT_FOR_REQUEST_RESPONSE)
        fsm.add_transition(source=self.WAIT_FOR_REQUEST_RESPONSE, dest=self.REFUSED_STATE)
        fsm.add_transition(source=self.WAIT_FOR_REQUEST_RESPONSE, dest=self.WAIT_FOR_DATA_STATE)
        fsm.add_transition(source=self.WAIT_FOR_DATA_STATE, dest=self.PROCESS_DATA_STATE)

        self.add_behaviour(fsm)

##keys.py
import os
from dotenv import load_dotenv
load_dotenv()

XMPP_ID_ONE=os.getenv('XMPP_ID_ONE')

```

```
XMPP_PASSWORD_ONE=os.getenv('XMPP_PASSWORD_ONE')
XMPP_ID_TWO=os.getenv('XMPP_ID_TWO')
XMPP_PASSWORD_TWO=os.getenv('XMPP_PASSWORD_TWO')
XMPP_ID_THREE=os.getenv('XMPP_ID_THREE')
XMPP_PASSWORD_THREE=os.getenv('XMPP_PASSWORD_THREE')
```

```
assert XMPP_ID_ONE is not None and XMPP_ID_ONE != "", "XMPP id ONE environment variable is not defined!"
assert XMPP_PASSWORD_ONE is not None and XMPP_PASSWORD_ONE != "", "XMPP password ONE environment
variable is not defined!"
assert XMPP_ID_TWO is not None and XMPP_ID_TWO != "", "XMPP id TWO environment variable is not defined!"
assert XMPP_PASSWORD_TWO is not None and XMPP_PASSWORD_TWO != "", "XMPP password TWO
environment variable is not defined!"
assert XMPP_ID_THREE is not None and XMPP_ID_THREE != "", "XMPP id THREE environment variable is not
defined!"
assert XMPP_PASSWORD_THREE is not None and XMPP_PASSWORD_THREE != "", "XMPP password THREE
environment variable is not defined!"
```

```
##run_client_one.py
import spade
import asyncio
import keys
from client_agent import ClientAgent

async def main():
    for i in range(3):
        client_agent = ClientAgent(keys.XMPP_ID_TWO, keys.XMPP_PASSWORD_TWO, f"Client 2.{i}",
keys.XMPP_ID_ONE)

        await client_agent.start()
        await spade.wait_until_finished(client_agent)

if __name__ == "__main__":
    asyncio.run(main())
```

```
##run_client_two.py
import spade
import asyncio
import keys
from client_agent import ClientAgent

async def main():
    for i in range(3):
        client_agent = ClientAgent(keys.XMPP_ID_THREE, keys.XMPP_PASSWORD_THREE, f"Client 2.{i}",
keys.XMPP_ID_ONE)

        await client_agent.start()
        await spade.wait_until_finished(client_agent)

if __name__ == "__main__":
    asyncio.run(main())
```

```
##run_service.py
import spade
import asyncio
import keys
from service_agent import ServiceAgent
```



```

async def main():
    service_agent = ServiceAgent(keys.XMPP_ID_ONE, keys.XMPP_PASSWORD_ONE, "Service")
    await service_agent.start()

    await spade.wait_until_finished(service_agent)

if __name__ == "__main__":
    asyncio.run(main())

##service_agent.py
import random
import asyncio

from spade.template import Template
from spade.agent import Agent
from spade.behaviour import CyclicBehaviour
from spade.message import Message

request_data_template = Template()
request_data_template.set_metadata("performative", "request")
request_data_template.set_metadata("language", "FIFA-SL")
request_data_template.set_metadata("ontology", "REQUEST-DATA")

agree_response_template = Template()
agree_response_template.set_metadata("performative", "agree")
agree_response_template.set_metadata("language", "FIFA-SL")
agree_response_template.set_metadata("ontology", "AGREE")

refuse_response_template = Template()
refuse_response_template.set_metadata("performative", "refuse")
refuse_response_template.set_metadata("language", "FIFA-SL")
refuse_response_template.set_metadata("ontology", "REFUSE")

data_response_template = Template()
data_response_template.set_metadata("performative", "inform")
data_response_template.set_metadata("language", "FIFA-SL")
data_response_template.set_metadata("ontology", "DATA")

class ServiceAgent(Agent):
    # Visual(in console) agent name. Different from jid
    name = "Unknown receiver"

    # How long we wait for message? In seconds
    message_timeout = 10

    def __init__(self, jid: str, password: str, name: str):
        super().__init__(jid, password)
        self.name = name

    class ReceiveBehaviour(CyclicBehaviour):
        async def send_data(self, to_id, value):
            random_time = random.randint(0, 100) / 50

            print(f"[ASYNC] {self.agent.name}: Schedule send message to {to_id} with value {value} after {random_time} seconds")
            await asyncio.sleep(random_time)

```

```

message = Message(to=to_id)
message.body = str(value)
message.set_metadata("language", "FIFA-SL")
message.set_metadata("performative", "inform")
message.set_metadata("ontology", "DATA")

assert data_response_template.match(message), "Data message should match data template"
await self.send(message)
print(f"[ASYNC] {self.agent.name}: Send data to {to_id}. Value is {value}")

async def run(self):
    print(f"\n{self.agent.name}: Waiting for messages!")
    received_message = await self.receive(timeout=self.agent.message_timeout)

    if not received_message:
        print(f"{self.agent.name}: Bye!")
        await self.agent.stop()
        return

    assert request_data_template.match(received_message), "Service agent receive only request data
messages"
    print(f"{self.agent.name}: Message from \"{received_message.sender}\" received!")

    random_value = random.randint(0, 10)
    message = Message(to=str(received_message.sender))
    message.body = "Request response"
    message.set_metadata("language", "FIFA-SL")

    if random_value > 8:
        message.set_metadata("performative", "refuse")
        message.set_metadata("ontology", "REFUSE")
        assert refuse_response_template.match(message), "Refuse message should match refuse template"
        await self.send(message)
        print(f"{self.agent.name}: Send refuse message to {message.to}")
        return

    message.set_metadata("performative", "agree")
    message.set_metadata("ontology", "AGREE")
    assert agree_response_template.match(message), "Agree message should match agree template"
    await self.send(message)
    print(f"{self.agent.name}: Send agree message to {message.to}")

    asyncio.create_task(self.send_data(str(received_message.sender), random_value))

async def setup(self):
    print(f"{self.name}: Hi!")

    self.add_behaviour(self.ReceiveBehaviour())

```

5 Висновки

Я ознайомився з поняттям складної поведінки агентів (Complex Behaviours) у мультиагентних системах. Навчився використовувати різні типи поведінки в SPADE: послідовну (Sequential), циклічну (Cyclic), умовну (FSM) та таймерну (Periodic). Реалізував агентів, які можуть адаптувати свою поведінку залежно від умов середовища.